

합성곱 신경망(CNN)

한경수
성결대학교 컴퓨터공학과

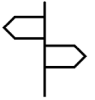
* 본 자료는 머신러닝 수업 수강 용도로만 사용이 가능함.
* 본 자료는 대부분 수업의 교재인 아래 도서를 기반으로 작성되었음.
한경수 머신러닝(3판), 오렐리안 제롬 지음, 박해선 옮김, 한빛미디어, 2023

학습 목표



- ◆ 공학 기술 및 도구 활용 역량
 - 합성곱 신경망(CNN)의 작동 원리를 설명할 수 있다.
- ◆ 요구 이해 및 설계 역량
 - 머신러닝 라이브러리를 이용하여 CNN 모델을 학습하고 학습 결과를 적용할 수 있다.

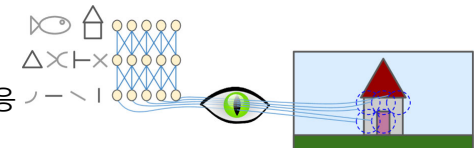
Introduction



- ◆ 머신러닝 기반으로 시각적 인식 시스템을 만들려면?
- ◆ 인간이나 다른 동물들은 어떻게 인식하는가?
 - 대뇌의 시각 피질(visual cortex) 연구
- ◆ 합성곱 신경망(CNN)
 - convolutional neural network
 - 컨볼루션 신경망
 - 복잡한 이미지 처리 문제에서 인간을 능가하는 성능 달성

시각 피질(Visual Cortex) 구조

- ◆ 시각 피질 안의 많은 뉴런이 작은 국부 수용장(local receptive field)을 가짐
 - 뉴런들이 시야의 제한된 영역 안에 있는 시각 자극에만 반응함
 - 뉴런들의 수용장들은 서로 겹칠 수 있고, 이들을 합치면 전체 시야를 감싸게 됨
 - 동일한 수용장을 가지는 뉴런들이라도 다른 각도의 선분에 반응
 - 수평선, 수직선, 대각선, ...
 - 어떤 뉴런들은 더 큰 수용장을 가짐
 - 저수준 패턴들이 조합된 더 복잡한 패턴들에 반응
- ◆ 고수준 뉴런들이 이웃한(직전의) 저수준 뉴런들의 출력에 기반한다는 아이디어 도출

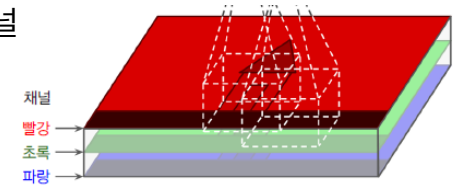


CNN

- ◆ 동물의 시각피질(visual cortex) 구조에서 영감을 받아 만들어진 딥러닝 신경망 모델
 - 계층적 특징 추출
 - 시각 자극이 1차 시각피질을 통해서 처리된 다음, 2차 시각피질을 경유하여, 3차 시각피질 등 여러 영역을 통과하여 계층적인 정보처리
- ◆ 정보가 계층적으로 처리되어 가면서 점차 추상적인 특징이 추출되어 시각 인식

이미지/사운드 클립 데이터의 특징

- ◆ 다차원 배열로 저장
- ◆ 순서가 중요한 하나 이상의 축이 있음
 - 이미지: 높이/폭 축, 사운드 클립: 시간 축
- ◆ 채널 축을 통해 데이터를 다양한 관점으로 접근
 - 컬러 이미지: R/G/B 채널
 - 스테레오 오디오 트랙: 좌/우 채널

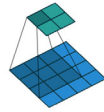


합성곱(Convolution)

합성곱 연산

- ◆ 순서 정보를 유지한 선형 변환
- ◆ Sparse함
 - 하나의 출력에 몇 개의 입력만 관여함
- ◆ 파라미터를 재사용
 - 동일한 가중치가 입력 데이터의 여러 위치에 적용됨

합성곱 연산



- ◆ 입력 특성 맵(feature map)을 커널(kernel)이 슬라이딩하여 지나가면서 출력 특성 맵을 산출함
- ◆ 각 위치에서, 커널의 각 원소와 오버랩되는 입력의 원소가 곱해져서 그 합이 현재 위치의 출력 값이 됨



한경수

출처1: https://github.com/vdumoulin/conv_arithmetic
출처2: 인공지능: 튜링 테스트에서 딥러닝까지, 이진명 지음, 생능출판사, 2018, p.266

9

합성곱 연산: 예1

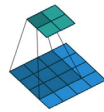


한경수

출처: Dumoulin et al., "A Guide to Convolution Arithmetic for Deep Learning", 2018, <https://arxiv.org/abs/1603.07285>

10

합성곱 연산



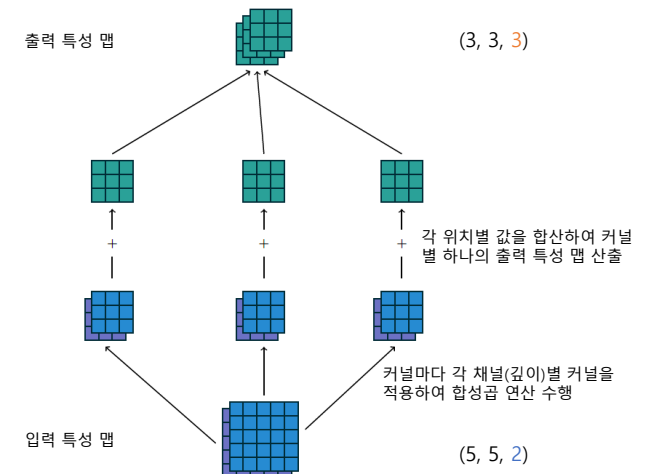
- ◆ 이 과정을 다양한 커널을 사용하여 반복함으로써, 원하는 개수만큼의 출력 특성 맵을 구성할 수 있음
- ◆ 입력 특성 맵이 여러(n) 개라면,
 - 각 특성 맵이 개별적인 커널과 합성곱 수행
 - 결과 특성 맵들(n 개)을 원소별로 더하여 출력 특성 맵(1개) 산출
- ◆ 출력 특성 맵 개수 = 커널 개수

한경수

11

합성곱 연산: 예2

입력 특성 맵: 2개
커널: 3개 (크기: 3*3)



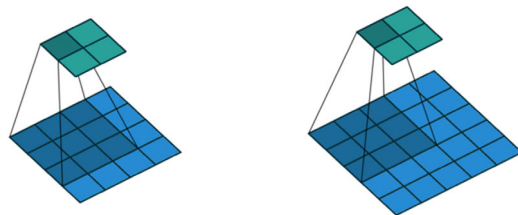
출처: Dumoulin et al., "A Guide to Convolution Arithmetic for Deep Learning", 2018, <https://arxiv.org/abs/1603.07285>

한경수

12

스트라이드(Stride)

- ◆ 다음 컨볼루션 연산을 위해 커널을 이동시키는 칸 수
 - 한 수용장과 다음 수용장 사이의 간격

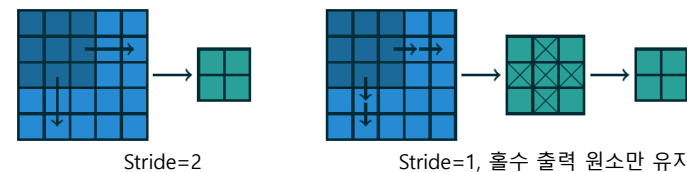


Stride=1
(4, 4) → (2, 2)

Stride=2
(5, 5) → (2, 2)

스트라이드(Stride)

- ◆ 서브샘플링(subsampling)의 한 형태
 - 얼마나 많은 출력을 유지하는지로 해석할 수도 있음

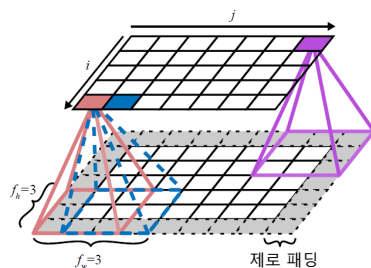


Stride=2

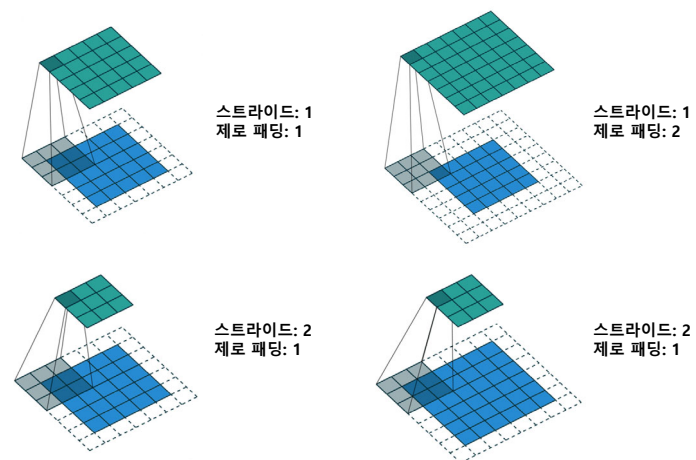
Stride=1, 홀수 출력 원소만 유지

제로 패딩(Zero Padding)

- ◆ 컨볼루션 결과의 크기를 조정하기 위해 입력의 주위에 0을 추가하는 연산
- ◆ 특성 맵의 높이/너비를 이전 층과 같게 하기 위해 제로 패딩을 수행하는 것이 일반적임



스트라이드와 제로 패딩



스트라이드: 1
제로 패딩: 1

스트라이드: 1
제로 패딩: 2

스트라이드: 2
제로 패딩: 1

스트라이드: 2
제로 패딩: 2

합성곱 연산: 출력 특성 맵의 크기

◆ 관계식

$$o = \left\lfloor \frac{i+2p-k}{s} \right\rfloor + 1$$

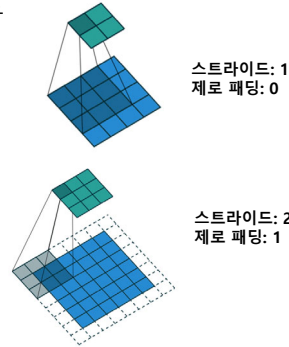
- o : 출력 크기 i : 입력 크기 s : 스트라이드
- p : 제로 패딩 크기 k : 커널 크기

◆ 예1: 스트라이드 1, 제로 패딩 0

$$o = \left\lfloor \frac{4+2\cdot0-3}{1} \right\rfloor + 1 = 1 + 1 = 2$$

◆ 예2: 스트라이드 2, 제로 패딩 1

$$o = \left\lfloor \frac{6+2\cdot1-3}{2} \right\rfloor + 1 = 2 + 1 = 3$$

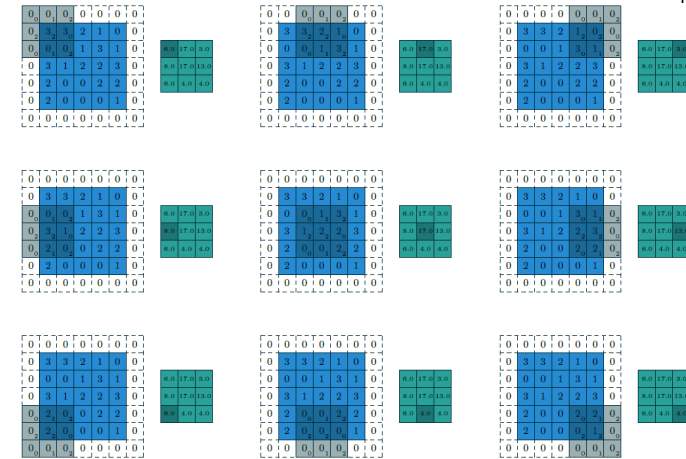


한경수

17

합성곱 연산: 예3

5*5 입력,
3*3 커널,
2*2 스트라이드,
1*1 제로 패딩,



한경수

출처: Dumoulin et al., "A Guide to Convolution Arithmetic for Deep Learning", 2018. 18
<https://arxiv.org/abs/1603.07285>

제로 패딩 옵션

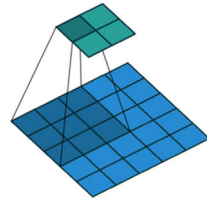
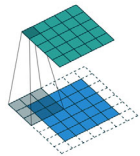
◆ 텐서플로우의 제로 패딩 옵션

- valid: 제로 패딩 미적용
- 커널이 유효한 입력 맵 밖으로 나가지 않음

$$o = \left\lfloor \frac{i+2p-k}{s} \right\rfloor + 1 = \left\lfloor \frac{i-k}{s} \right\rfloor + 1$$

- same: 동일 패딩 적용

- 출력의 크기가 입력의 크기와 동일해지도록 필요한 만큼 입력 맵 가장자리 양 쪽에 각각 제로 패딩
- 단, 제로 패딩 크기(가장자리 한 쪽에 추가하는 개수)는 커널 크기 절반을 넘지 않도록 함; half padding
- $o = \lceil i/s \rceil$
- 스트라이드=1이면, 출력의 크기가 입력의 크기와 동일하게 유지될 수 있음
- 스트라이드=2이면, 출력 크기는 입력 크기의 절반



한경수

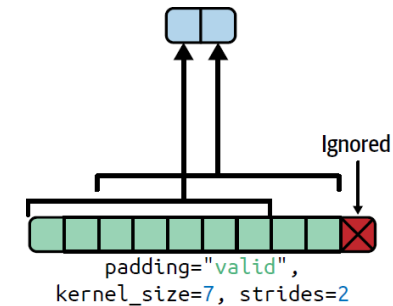
19

제로 패딩 옵션: valid 예

$$o = \left\lfloor \frac{i-k}{s} \right\rfloor + 1$$

- ◆ 입력 크기: 10, 커널 크기: 7, 스트라이드: 2

$$\text{출력 크기: } \left\lfloor \frac{10-7}{2} \right\rfloor + 1 = 1 + 1 = 2$$



한경수

20

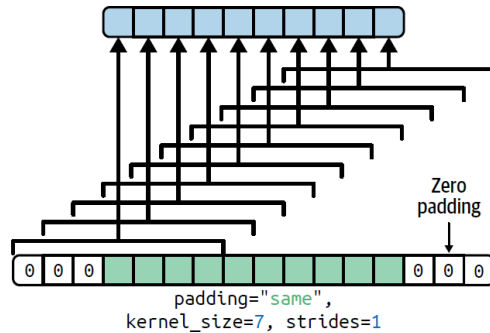
제로 패딩 옵션: same 예1

$$o = \lceil i/s \rceil$$

- ◆ 입력 크기: 10, 커널 크기: 7, 스트라이드: 1
- 출력 크기: 10

```
# 총 패딩 양 계산:
if i%s == 0:
    pad = max( k-s, 0 )
else
    pad = max( k - (i%s), 0 )

# 총 패딩 양이 홀수인 경우,
# 오른쪽을 하나 더 패딩
left_pad = pad // 2
right_pad = pad - left_pad
```



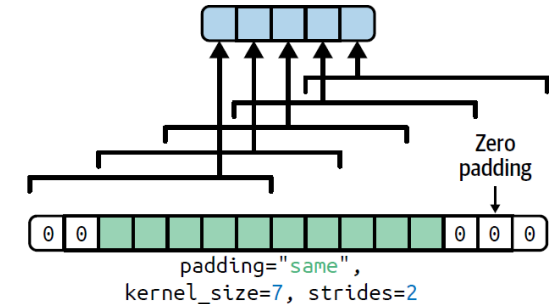
제로 패딩 옵션: same 예2

$$o = \lceil i/s \rceil$$

- ◆ 입력 크기: 10, 커널 크기: 7, 스트라이드: 2
- 출력 크기: $\lceil \frac{10}{2} \rceil = \lceil 5 \rceil = 5$

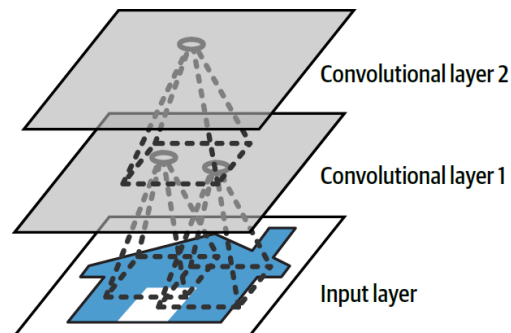
```
# 총 패딩 양 계산:
if i%s == 0:
    pad = max( k-s, 0 )
else
    pad = max( k - (i%s), 0 )

# 총 패딩 양이 홀수인 경우,
# 오른쪽을 하나 더 패딩
left_pad = pad // 2
right_pad = pad - left_pad
```



합성곱 층

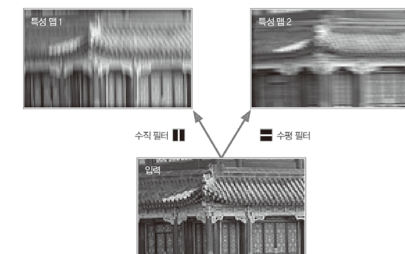
- ◆ 첫번째 합성곱 층의 뉴런
 - 입력 이미지의 모든 픽셀에 각각 연결되지 않고, 자신의 수용장 안에 있는 픽셀에만 연결됨
- ◆ 두번째 합성곱 층의 뉴런
 - 첫번째 층의 작은 사각 영역에만 연결됨
- ◆ 첫번째 은닉층에서는 작은 저수준 특성으로 조합되고, 그 다음 큰 조합된 고수준 특성으로 조합



합성곱 층: 필터(커널)

0	1	2
2	2	0
0	1	2

- ◆ 하나의 필터는 하나의 특성 맵을 만들
 - 필터를 가장 크게 활성화시키는 이미지 영역 강조
- ◆ 학습하는 동안 합성곱 층이 해당 문제에 가장 유용한 필터를 자동으로 학습하고, 상위 층은 이들을 결합하여 더 복잡한 패턴 학습

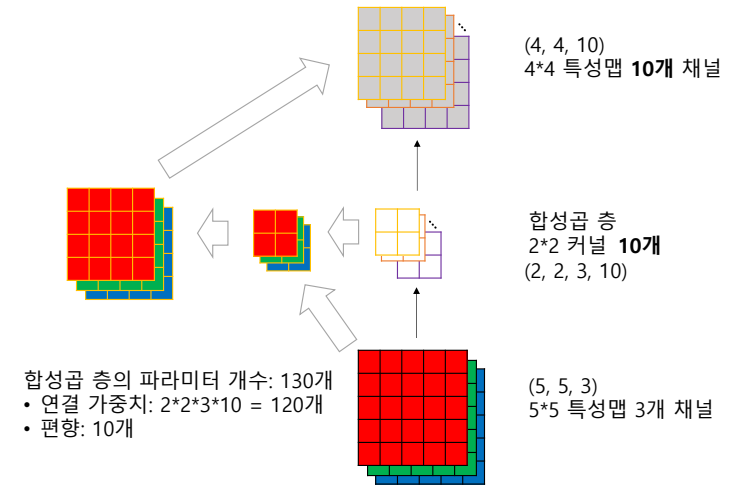


합성곱 층: 여러 특성 맵 쌓기

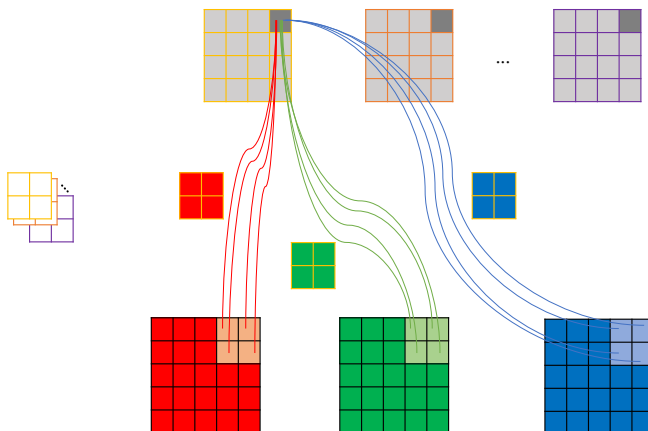
- ◆ 하나의 합성곱 층은 3D로 표현할 수 있음
 - 여러 필터를 가짐
 - 각 필터는 하나의 특성 맵을 출력함
- ◆ 특성 맵의 각 픽셀당 하나의 뉴런이 있음
- ◆ 하나의 특성 맵 내의 모든 뉴런은 같은 파라미터(커널 가중치)를 공유
 - 다른 특성 맵에 있는 뉴런은 다른 파라미터 사용
- ◆ 한 뉴런의 수용장은 이전 층의 모든 특성 맵에 걸쳐 확장됨
 - 예: 입력 층인 경우 모든 채널의 특성 맵에 대해 합성곱 연산 수행
- ◆ 하나의 합성곱 층은 학습 가능한 여러 필터를 입력에 동시에 적용하여 입력에 있는 **여러 특성을 감지**함

예: 128개 필터 → (H, W, 128)
입력 특성 맵의 각 채널마다 필터 가중치가 별도로 학습됨.

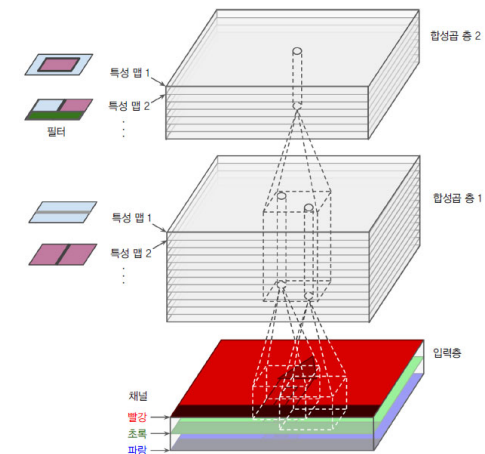
합성곱 층: 여러 특성 맵 쌓기 - 예



합성곱 층: 여러 특성 맵 쌓기 - 예



합성곱 층: 여러 특성 맵 쌓기



풀링(Pooling)

한경수

29

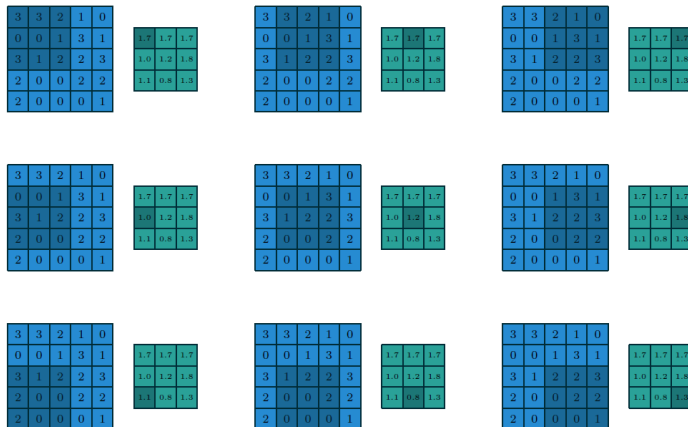
풀링 연산

- ◆ 일부 영역을 요약하는 함수를 사용하여 특성 맵의 크기를 줄이는 연산
 - 풀링 함수 예: 평균값, 최대값
 - 일부 영역을 통합하여 하나의 대푯값으로 대체
- ◆ 윈도우를 입력 값 위로 슬라이딩하면서 윈도우의 내용을 풀링 함수의 입력으로 전달

한경수

30

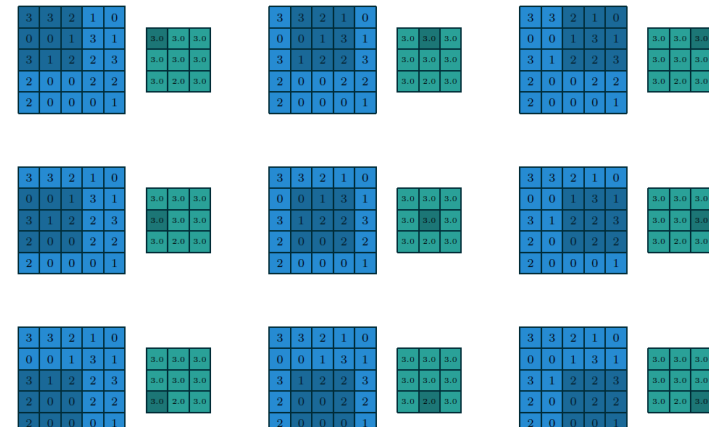
풀링 연산 예: 평균 풀링(Average Pooling)



한경수

출처: Dumoulin et al., "A Guide to Convolution Arithmetic for Deep Learning", 2018. <https://arxiv.org/abs/1603.07285> 31

풀링 연산 예: 최대 풀링(Max Pooling)



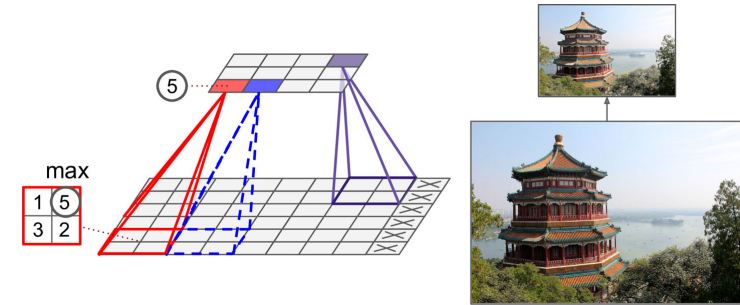
한경수

출처: Dumoulin et al., "A Guide to Convolution Arithmetic for Deep Learning", 2018. <https://arxiv.org/abs/1603.07285> 32

풀링 층

- ◆ 계산량, 메모리 사용량, 파라미터 수를 줄이기 위해 입력 이미지를 축소;
서브샘플링(subsampling)
→ 과대적합 위험 감소
- ◆ 풀링 층의 뉴런(풀링 뉴런)은 **가중치가 없음**
 - 최대나 평균 같은 집계 함수를 사용해 입력을 결합하는 것이 전부임
- ◆ 풀링 층은 보통 모든 입력 **채널에 독립적으로 적용됨**
 - 출력 깊이 = 입력 깊이

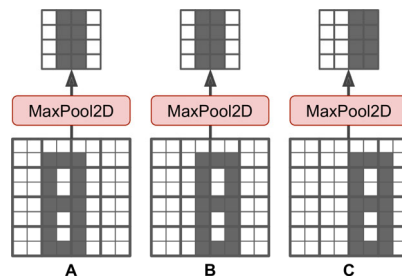
최대 풀링 층



2 × 2 풀링 커널; 스트라이드 2; 패딩 없음

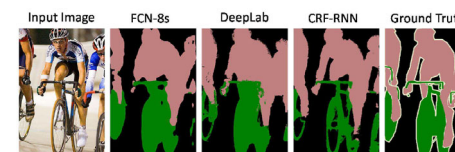
최대 풀링: 이동 불변성(Translation Invariance)

- ◆ 작은 위치 변화에도 어느 정도의 불변성 제공
- ◆ CNN에서 몇 개 층마다 최대 풀링 층을 추가하면 전체적으로 어느 정도의 이동 불변성 획득 가능
 - 회전, 확대/축소에 대해서는 약간의 불변성 제공
- ◆ 분류 작업처럼 예측이 작은 위치 변화에 영향을 받지 않는 경우 유용할 수 있음



최대 풀링

- ◆ 매우 파괴적
 - 많은 입력 값을 잃게 됨
 - 예: 2*2 커널, 스트라이드 2 → 75% 손실
- ◆ 불변성이 바람직하지 않은 애플리케이션도 있음
 - 예: 시맨틱 분할(픽셀이 속한 객체에 따라 픽셀을 각 객체 클래스로 분류하는 작업)



입력의 작은 변화가 출력에서도 그에 상응하는 작은 변화로 이어져야 함

최대 풀링과 평균 풀링

- ◆ 최대 풀링 층이 일반적으로 더 성능 좋음
 - 평균 계산이 최대값 계산보다 정보 손실이 적지만,
 - 최대 풀링은
 - 의미 없는 것은 모두 제거하고
 - 가장 큰 특성만 유지함
 - 다음 층이 조금 더 명확한 신호로 작업
 - 평균 풀링보다 더 강력한 이동 불변성 제공
 - 연산 비용이 조금 덜 듬

전역 평균 풀링 층

- ◆ Global average pooling layer
- ◆ 각 특성 맵마다 평균 계산
 - 입력과 동일한 크기의 풀링 커널을 사용하는 것과 같음
 - 각 샘플의 각 특성 맵마다 하나의 평균 값을 출력
- ◆ 출력층에 유용할 수 있음

전치 합성곱 (Transposed Convolution)

전치 합성곱 연산

- ◆ 합성곱 연산의 반대 방향으로 변환
 - 합성곱: 큰 데이터 → 작은 데이터
 - 전치 합성곱: 작은 데이터 → 큰 데이터
 - 입출력 연결 패턴은 그대로 유지
- ◆ 주요 용도
 - 합성곱 오토인코더의 디코딩 층
 - 특성맵을 더 높은 차원의 공간으로 투영;
업샘플링(upsampling)

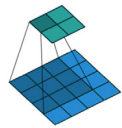
전치 합성곱 연산

i : 입력 크기
 o : 출력 크기
 k : 커널 크기
 s : 스트라이드
 p : 패딩 크기

◆ 전치 합성곱을 합성곱 연산으로 해석

- i, o, k, s, p 로 표현되는 합성곱 연산의 역방향에 해당하는 전치 합성곱 연산은 다음과 같은 i', o', k', s', p' 로 표현되는 합성곱으로 간주할 수 있음

- $i' = o, k' = k, s' = 1, p' = k - p - 1$
- i' : 각 입력 유닛 사이에 $(s - 1)$ 개의 제로를 추가한 확장된 입력의 크기
- $o' = s(i' - 1) + k - 2p$

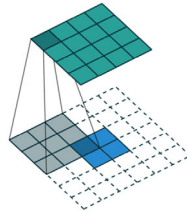


합성곱

$i = 4,$
 $k = 3,$
 $s = 1,$
 $p = 0,$
 $o = 2$

$i' = 2,$
 $k' = 3,$
 $s' = 1,$
 $p' = 3 - 0 - 1 = 2,$
 $i' = 6,$
 (유닛 사이에 0개의 제로 추가)

$o' = 1 \times (2 - 1) + 3 - 2 \times 0 = 4$



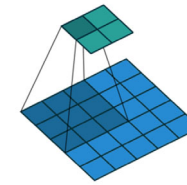
전치 합성곱

출처: https://github.com/vdumoulin/conv_arithmetic

CNN 구조

$i' = o, k' = k, s' = 1, p' = k - p - 1$
 i' : 각 입력 유닛 사이에 $(s - 1)$ 개의 제로를
 추가한 확장된 입력의 크기
 $o' = s(i' - 1) + k - 2p$

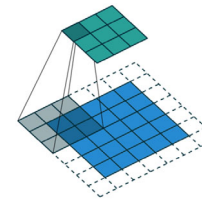
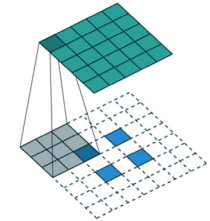
전치 합성곱 연산 예



$k = 3, s = 2, p = 0$

$i' = 2,$
 $k' = 3,$
 $s' = 1,$
 $p' = 3 - 0 - 1 = 2,$
 $i' = 7,$
 (유닛 사이에 1개의 제로 추가)

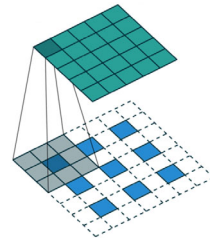
$o' = 2 \times (2 - 1) + 3 - 2 \times 0 = 5$



$k = 3, s = 2, p = 1$

$i' = 3,$
 $k' = 3,$
 $s' = 1,$
 $p' = 3 - 1 - 1 = 1,$
 $i' = 7,$
 (유닛 사이에 1개의 제로 추가)

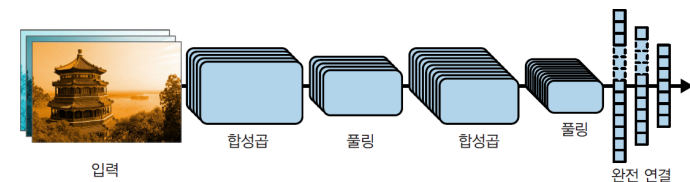
$o' = 2 \times (3 - 1) + 3 - 2 \times 1 = 5$



출처: https://github.com/vdumoulin/conv_arithmetic

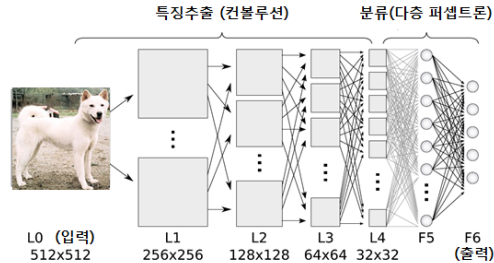
CNN 구조

- ◆ 네트워크를 통과하여 진행할수록 이미지는 점점 작아지지만, 점점 더 깊어짐
 - 특성 맵이 더 많아짐; 합성곱 층 때문
 - 합성곱 층에 너무 큰 크기의 커널은 사용하지 않는 것이 좋음
 - 5×5 커널 보다는 3×3 커널 두 층을 쌓는 것이 나음
 - 파라미터 계산량이 적고 성능이 더 좋음
 - 단, 첫번째 합성곱 층에는 채널(깊이)이 3뿐이므로 큰 크기의 커널 사용 가능; 스트라이드는 2 이상



CNN 구조

- ◆ 전반부: 특성 추출
 - 합성곱 연산, 풀링 연산
- ◆ 후반부: 특성들을 이용하여 분류
 - 다층 퍼셉트론(MLP)



한경수

출처: 인공지능: 튜링 테스트에서 딥러닝까지, 이진영 지음, 생능출판사, 2018, p.265 45

LeNet-5

$$o = \left\lfloor \frac{i + 2p - k}{s} \right\rfloor + 1$$

- ◆ Yann Lecun(1998)
 - <http://yann.lecun.com/exdb/lenet/>
- ◆ 손글씨 숫자 인식(MNIST)에 널리 사용됨

층	종류	특성 맵	크기	커널 크기	스트라이드	활성화 함수
출력	완전연결	-	10	-	-	RBF
F6	완전연결	-	84	-	-	tanh
C5	합성곱	120	1×1	5×5	1	tanh
S4	평균 풀링	16	5×5	2×2	2	tanh
C3	합성곱	16	10×10	5×5	1	tanh
S2	평균 풀링	6	14×14	2×2	2	tanh
C1	합성곱	6	28×28	5×5	1	tanh
입력	입력	1	32×32	-	-	-

한경수

MNIST 이미지 28×28에 zero padding → 32×32; 나머지 부분에는 padding 미사용

46

AlexNet

- ◆ Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton @Toronto Univ.(2012)

층	종류	특성 맵	크기	커널 크기	스트라이드	패딩*	활성화 함수
출력	완전연결	-	1000	-	-	-	Softmax
F10	완전연결	-	4096	-	-	-	ReLU
F9	완전연결	-	4096	-	-	-	ReLU
F8	최대 풀링	256	6×6	3×3	2	valid	-
C7	합성곱	256	13×13	3×3	1	same	ReLU
C6	합성곱	384	13×13	3×3	1	same	ReLU
C5	합성곱	384	13×13	3×3	1	same	ReLU
S4	최대 풀링	256	13×13	3×3	2	valid	-
C3	합성곱	256	27×27	5×5	1	same	ReLU
S2	최대 풀링	96	27×27	3×3	2	valid	-
C1	합성곱	96	55×55	11×11	4	valid	ReLU
입력	입력	3(RGB)	227×227	-	-	-	-

한경수

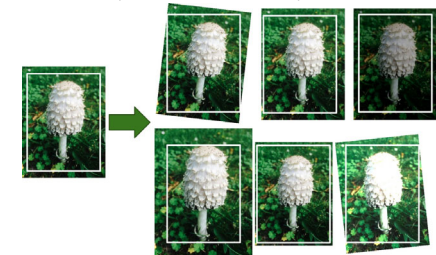
* same: zero padding 적용, valid: 미적용

47

AlexNet

- ◆ 과대적합 줄이기 위해 두 가지 규제 기법 사용
 - F9, F10의 출력에 드롭아웃을 50% 비율로 적용
 - 데이터 증식(data augmentation) 수행
 - 학습 샘플을 인공적으로 생성하여 학습 집합 크기 증대
 - 학습 이미지를 랜덤하게 여러 간격으로 이동, 수평으로 뒤집기, 조명 조건 바꾸기 등

층	종류	특성 맵	크기
출력	완전연결	-	1000
F10	완전연결	-	4096
F9	완전연결	-	4096
F8	최대 풀링	256	6×6
C7	합성곱	256	13×13
...			



한경수

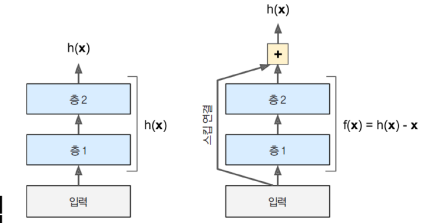
48

ResNet

- 잔차 네트워크(residual network); 2015
- 152개 층으로 구성된 극도로 깊은 CNN
 - 34개, 50개, 101개 층을 가진 변종도 있음
- 모델은 더 깊게; 파라미터는 더 적게 사용하는 트렌드
- 깊은 네트워크 학습을 가능케하는 핵심 요소
 - 스킵 연결(skip connection); 숏컷 연결(shortcut connection)
 - 어떤 층에 주입되는 신호가 상위 층의 출력에도 더해짐

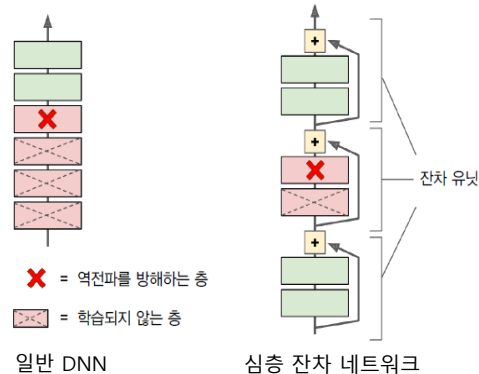
ResNet: 잔차 학습(Residual Learning)

- 일반 신경망
 - 목적 함수 $h(\mathbf{x})$ 모델링
- 잔차 학습
 - $f(\mathbf{x}) = h(\mathbf{x}) - \mathbf{x}$ 를 모델링
 - 초기에 입력과 거의 같은 값을 출력
 - 항등 함수(identity function)
 - 신경망 가중치 초기값이 0에 가깝기 때문
 - 목적 함수가 항등 함수에 매우 가깝다면 학습 속도가 매우 빨라짐

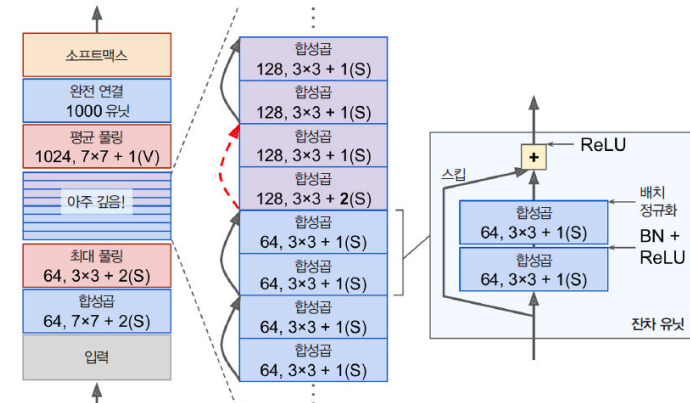


잔차 학습

- 스킵 연결을 많이 추가하면
 - 일부 층이 아직 학습을 시작하지 않았더라도 네트워크는 학습을 진행해 갈 수 있음
 - 신호가 네트워크 전역에 쉽게 전달될 수 있음
- 심층 잔차 네트워크
 - 잔차 유닛을 쌓은 것
 - 잔차 유닛(residual unit; RU): 스킵 연결을 가진 작은 신경망



ResNet 구조

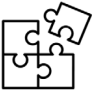


ResNet-34, ResNet-152: 34/152개 층으로 이루어진 ResNet

기타 참고

- ◆ YOLO
 - <https://docs.ultralytics.com/>
 - 객체 탐지 및 트래킹, 이미지 분할, 이미지 분류, 자세 추정 등
- ◆ OpenCV
 - <https://opencv.org/>
 - 이미지/영상 처리, 그래픽스 등 컴퓨터 비전 관련

정리



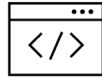
- ◆ 합성곱
- ◆ 풀링
- ◆ 전치 합성곱
- ◆ CNN 구조
 - LeNet-5
 - AlexNet: 데이터 증식; LRN
 - ResNet: 스킵 연결; 잔차 학습; 잔차 유닛



정리 문제 풀이

정리 문제: CNN

- ◆ 이미지 분류 문제에서 완전 연결 DNN보다 CNN을 사용한 경우 성능이 더 좋을까? 이유는?



실습

실습: CNN

- ◆ 합성곱 층
 - `keras.layers.Conv2D`
- ◆ 풀링 층
 - 최대 풀링: `keras.layers.MaxPool2D`
 - 평균 풀링: `keras.layers.AvgPool2D`
 - 전역 평균 풀링: `keras.layers.GlobalAvgPool2D`
- ◆ 전치 합성곱 층
 - `keras.layers.Conv2DTranspose`